



01 Questions 1–5 — Extract Date Parts

Extract individual pieces from a date column — the day name, month name, month number, year, and day of the month.

QUESTION 01

Day Name from Order Date

Table • orders

order_id	customer_id	order_date
1001	101	2026-05-01
1002	102	2026-05-02
1003	103	2026-05-03
1004	104	2026-05-04
1005	105	2026-05-05

TASK

Return each order with the day name of the order date.

USE

DAYNAME • AS

EXPECTED OUTPUT COLUMNS

order_id, customer_id, order_date, day_name

WRITE YOUR SQL HERE

```
SELECT order_id,  
       customer_id,  
       order_date,  
       DAYNAME(order_date) AS day_name  
FROM retail.datefunc.orders;
```

QUESTION 02

Month Name from Signup Date

Table • customer_signups



customer_id	customer_name	signup_date
201	John	2026-01-15
202	Mary	2026-02-20
203	Peter	2026-03-05
204	Sarah	2026-04-18
205	Thabo	2026-05-30

T A S K

Return each customer with the name of the month in which they signed up.

U S E

MONTHNAME AS

EXPECTED OUTPUT COLUMNS

customer_id, customer_name, signup_date, signup_month_name

WRITE YOUR SQL HERE

```
SELECT customer_id, customer_name, signup_date,  
       MONTHNAME(signup_date) AS signup_month_name  
FROM retail.datefunc-customer-signups;
```

QUESTION 03**Month Number from Sale Date**

Table - sales

sale_id	product_name	sale_date	amount
S001	Laptop	2026-01-10	12000
S002	Mouse	2026-02-15	250
S003	Keyboard	2026-03-20	700
S004	Monitor	2026-04-25	3500
S005	Desk	2026-05-30	2000

T A S K

Return each sale with the month number extracted from sale_date.



USE

MONTH . AS

EXPECTED OUTPUT COLUMNS

sale_id, product_name, sale_date, sale_month

WRITE YOUR SQL HERE

```
SELECT sale_id, product_name, sale_date,  
       MONTH(sale_date) AS sale_month
```

```
FROM retail.datefunc.sales;
```

QUESTION 04

Year from Transaction Date

Table . transactions

transaction_id	customer_id	transaction_date	amount
T001	101	2024-12-15	500
T002	102	2025-01-20	1200
T003	103	2025-06-10	800
T004	104	2026-02-05	1500
T005	105	2026-05-25	2000

TASK

Return each transaction with the year extracted from transaction_date.

USE

YEAR . AS

EXPECTED OUTPUT COLUMNS

transaction_id, customer_id, transaction_date, transaction_year

WRITE YOUR SQL HERE

```
SELECT transaction_id, customer_id, transaction_date,  
       YEAR(transaction_date) AS transaction_year  
FROM retail.datefunc.transactions;
```



QUESTION 05

Day of Month from Delivery Date

Table - deliveries

delivery_id	customer_id	delivery_date
D001	101	2026-05-01
D002	102	2026-05-08
D003	103	2026-05-15
D004	104	2026-05-22
D005	105	2026-05-29

TASK

Return each delivery with the day of the month extracted from delivery_date.

USE

DAY AS

EXPECTED OUTPUT COLUMNS

delivery_id, customer_id, delivery_date, day_of_month

WRITE YOUR SQL HERE

→ inserted back ticks because Databricks confuses this with the customer-id in transactions

```
SELECT delivery_id, 'customer-id', delivery_date,  
       DAY(delivery_date) AS day-of-month  
FROM retail.datefunc.deliveries;
```

02

Questions 6-10 — Current Date, Convert & Format

Use the current date, convert text strings to date values, and format dates for display.

QUESTION 06

Add Today's Date

Table - employees

employee_id	employee_name	department
301	Nandi	Sales
302	Brian	IT
303	Lerato	Finance
304	Sipho	HR
305	Aisha	Marketing

TASK

Return all employees and add a column showing today's date.

USE

`CURRENT_DATE()` AS

EXPECTED OUTPUT COLUMNS

employee_id, employee_name, department, today_date

WRITE YOUR SQL HERE

```
SELECT employee_id, employee_name, department,  
       CURRENT_DATE AS today_date  
FROM retail.datefunc.employees;
```

QUESTION 07

Convert Text to Date

Table · online_orders

order_id	customer_id	order_date_text
4001	101	2026-01-15
4002	102	2026-02-20
4003	103	2026-03-25
4004	104	2026-04-10
4005	105	2026-05-05

TASK

Convert the order_date_text string column into a proper date column.

USE

TO_DATE · AS

EXPECTED OUTPUT COLUMNS

order_id, customer_id, order_date_text, order_date

WRITE YOUR SQL HERE

```
SELECT order_id, customer_id, TO_DATE (order_date_text) AS order_date
FROM retail-default.online_orders;
```

QUESTION 08

Format Date as Text

Table · payment_dates

payment_id	customer_id	payment_date
P001	101	2026-01-05
P002	102	2026-02-10
P003	103	2026-03-15
P004	104	2026-04-20
P005	105	2026-05-25

TASK



Format payment_date as a text string in the format YYYY-MM-DD.

USE

TO_CHAR AS

EXPECTED OUTPUT COLUMNS

payment_id, customer_id, payment_date, formatted_payment_date

WRITE YOUR SQL HERE

SELECT payment_id, customer_id, payment_date,
TO_CHAR(payment_date, 'yyy-MM-dd') AS formatted_payment_date
FROM retail.datefunc.payment_dates;
YYYY-MM-DD - returns ERROR on Databricks because date
format is case sensitive.

QUESTION 09

Days Since Last Purchase

Table: customer_purchases

customer_id	customer_name	last_purchase_date
501	John	2026-05-01
502	Mary	2026-05-10
503	Peter	2026-05-15
504	Sarah	2026-05-20
505	Thabo	2026-05-25

TASK

Calculate how many days have passed since each customer's last purchase. Use today's date.

USE

CURRENT_DATE() - DATEDIFF AS

EXPECTED OUTPUT COLUMNS

customer_id, customer_name, last_purchase_date, days_since_last_purchase

WRITE YOUR SQL HERE



```

SELECT customer_id, customer_name, last_purchase_date,
       DATEDIFF(current_date, last_purchase_date) AS days_since_last_purchase
FROM retail_datafunc.customer_purchases;

```

QUESTION 10

Expected Delivery Date

Table: shipping_orders

order_id	customer_id	order_date
6001	101	2026-05-01
6002	102	2026-05-03
6003	103	2026-05-05
6004	104	2026-05-07
6005	105	2026-05-09

TASK

Calculate the expected delivery date by adding 7 days to the order_date.

USE

DATEADD AS

EXPECTED OUTPUT COLUMNS

order_id, customer_id, order_date, expected_delivery_date

WRITE YOUR SQL HERE

```

SELECT order_id, customer_id, order_date,
       DATE_ADD(to_date(order_date), 7) AS expected_delivery_date
FROM retail_datafunc.shipping_orders;

```




03

Questions 11-15 — Multi-Extract & Date Filtering

Extract multiple date parts at once, filter rows by year or month, and use LAST_DAY and DATE_TRUNC.

QUESTION 11

Year, Month, Day Separately

Table • bookings

booking_id	customer_id	booking_date
B001	101	2026-01-12
B002	102	2026-02-18
B003	103	2026-03-22
B004	104	2026-04-09
B005	105	2026-05-27

TASK

Extract the year, month number, and day from booking_date into three separate columns.

USE

EXPECTED OUTPUT COLUMNS

booking_id, customer_id, booking_date, booking_year, booking_month, booking_day

WRITE YOUR SQL HERE

```
SELECT booking_id, customer_id, booking_date,  
       YEAR (booking_date) AS booking_year,  
       MONTH (booking_date) AS booking_month,  
       DAY (booking_date) AS booking_day  
FROM retail.datafunc.bookings;
```

QUESTION 12



Filter Orders from 2026

Table • yearly_orders

order_id	customer_id	order_date	amount
7001	101	2024-12-15	500
7002	102	2025-01-20	1200
7003	103	2025-06-10	800
7004	104	2026-02-05	1500
7005	105	2026-05-25	2000

TASK

Return only orders from the year 2026.

USE

YEAR • WHERE • AS

EXPECTED OUTPUT COLUMNS

order_id, customer_id, order_date, order_year, amount

WRITE YOUR SQL HERE

```
SELECT order_id, customer_id, order_date, amount,  
       YEAR (order_date) AS order_year  
FROM retail.datefunc.yearly_orders;  
WHERE YEAR (order_date) = 2026;
```

QUESTION 13

Filter Orders from March

Table • monthly_orders

order_id	customer_id	order_date	amount
8001	101	2026-01-15	500
8002	102	2026-02-20	1200
8003	103	2026-03-10	800
8004	104	2026-03-25	1500
8005	105	2026-05-30	2000



TASK

Return only orders placed in March (month number 3).

USE

MONTH • WHERE • AS

EXPECTED OUTPUT COLUMNS

order_id, customer_id, order_date, order_month, amount

WRITE YOUR SQL HERE

```
SELECT order_id, customer_id, order_date, amount,  
       MONTH(order_date) AS order_month  
FROM retail.datefunc.monthly_orders  
WHERE MONTH(order_date) = 03;
```

QUESTION 14

Last Day of the Month

Table • subscriptions

subscription_id	customer_id	start_date
SUB001	101	2026-01-10
SUB002	102	2026-02-15
SUB003	103	2026-03-20
SUB004	104	2026-04-25
SUB005	105	2026-05-30

TASK

Return the last day of the month for each subscription start date.

USE

LAST_DAY • AS

EXPECTED OUTPUT COLUMNS

subscription_id, customer_id, start_date, month_end_date

WRITE YOUR SQL HERE



```
SELECT subscription_id, customer_id, start_date,  
       LAST_DAY(start_date) AS month-end-date  
FROM retail.datefunc.subscriptions;
```

QUESTION 15

First Day of the Month

Table - campaign_sends

send_id	customer_id	send_date
C001	101	2026-01-12
C002	102	2026-02-18
C003	103	2026-03-22
C004	104	2026-04-09
C005	105	2026-05-27

TASK

Return the first day of the month for each campaign send date.

USE

DATE_TRUNC AS

EXPECTED OUTPUT COLUMNS

send_id, customer_id, send_date, month_start_date

WRITE YOUR SQL HERE

```
SELECT send_id, customer_id, send_date,  
       DATE_TRUNC('MONTH', send_date) AS month-start-date  
FROM retail.datefunc-campaign-sends;
```



04

Questions 16–20 — Advanced Date Work

The final five questions involve formatting, age calculations, CASE with dates, quarter extraction, and filtering by date difference.

QUESTION 16

Format as Month and Year

Table - invoice_dates

invoice_id	customer_id	invoice_date
INV001	101	2026-01-05
INV002	102	2026-02-10
INV003	103	2026-03-15
INV004	104	2026-04-20
INV005	105	2026-05-25

TASK

Format `invoice_date` as a text string showing month name and year. Example: January 2026.

USE

`TO_CHAR` - AS

EXPECTED OUTPUT COLUMNS

`invoice_id`, `customer_id`, `invoice_date`, `invoice_month_year`

WRITE YOUR SQL HERE

```
SELECT invoice_id, customer_id, invoice_date,
       TO_CHAR(invoice_date, 'MMMMM yyyy') AS invoice_month_year
FROM   retail.datefunc, invoice_dates;
```

QUESTION 17

Customer Age from Date of Birth



Table • customer_birthdays

customer_id	customer_name	date_of_birth
901	John	1998-05-10
902	Mary	1990-08-20
903	Peter	2002-03-15
904	Sarah	1985-12-01
905	Thabo	2000-07-30

TASK

Calculate each customer's age in years using their date of birth and today's date.

USE

CURRENT_DATE() • DATEDIFF • YEAR • AS

EXPECTED OUTPUT COLUMNS

customer_id, customer_name, date_of_birth, customer_age

WRITE YOUR SQL HERE

```
SELECT customer_id, customer_name, date_of_birth,
       DATEDIFF(CURRENT_DATE(), date_of_birth)/365 AS customer_age
FROM retail_datafunc.customer_birthdays;
```

QUESTION 18

Classify Weekend vs Weekday

Table • weekend_orders

order_id	customer_id	order_date
9001	101	2026-05-01
9002	102	2026-05-02
9003	103	2026-05-03
9004	104	2026-05-04
9005	105	2026-05-05

TASK



Classify each order as Weekend or Weekday based on the day name of the order date.
Saturday and Sunday = Weekend. All others = Weekday.

USE

DAYNAME • CASE • AS

EXPECTED OUTPUT COLUMNS

order_id, customer_id, order_date, day_name, day_type

WRITE YOUR SQL HERE

```
SELECT order_id, customer_id, order_date,  
       DAYNAME (order_date) AS day_name,  
       CASE  
         WHEN DAYNAME (order_date) IN ('Sun', 'Sat') THEN 'Weekend'  
         ELSE 'Weekday'  
       END AS day_type  
FROM retail_datefunc.weekend_orders;
```

QUESTION 19

Transactions by Quarter

Table • quarterly_transactions

transaction_id	customer_id	transaction_date	amount
Q001	101	2026-01-15	500
Q002	102	2026-03-20	1200
Q003	103	2026-04-10	800
Q004	104	2026-07-05	1500
Q005	105	2026-10-25	2000

TASK

Extract the quarter number from each transaction date.

USE

QUARTER • AS

EXPECTED OUTPUT COLUMNS

transaction_id, customer_id, transaction_date, transaction_quarter, amount

WRITE YOUR SQL HERE



```

SELECT transaction_id, customer_id, transaction_date, amount,
       QUARTER(transaction_date) AS transaction_quarter
FROM retail.datefunc.quarterly_transactions;

```

QUESTION 20

Orders Older Than 30 Days

Table - recent_orders

order_id	customer_id	order_date	amount
R001	101	2026-04-01	500
R002	102	2026-04-15	1200
R003	103	2026-05-01	800
R004	104	2026-05-10	1500
R005	105	2026-05-25	2000

TASK

Return only orders that are more than 30 days old from today. Calculate days_since_order using today's date.

USE

```
CURRENT_DATE() - DATEDIFF(ORDER_DATE, CURRENT_DATE) AS
```

EXPECTED OUTPUT COLUMNS

```
order_id, customer_id, order_date, days_since_order, amount
```

WRITE YOUR SQL HERE

```

SELECT order_id, customer_id, order_date, amount,
       DATEDIFF(current_date, order_date) AS days-since-order
FROM retail.datefunc.recent-orders
WHERE DATEDIFF(current_date, order_date) > 30;

```




05

Bonus Challenge

The bonus combines DATEDIFF, CASE, and CURRENT_DATE into a single customer segmentation query. Apply every concept from this guide in one question.

QUESTION BONUS

Customer Recency Segment

Table · customer_recency

customer_id	customer_name	last_purchase_date	total_spend
1001	John	2026-05-25	5000
1002	Mary	2026-05-10	2500
1003	Peter	2026-04-01	700
1004	Sarah	2026-02-15	15000
1005	Thabo	2025-12-20	300

TASK

Calculate days_since_last_purchase for each customer using CURRENT_DATE. Then use CASE to classify: ≤ 30 days = Active Customer, 31–90 days = At Risk Customer, > 90 days = Inactive Customer.

USE

CURRENT_DATE() · DATEDIFF · CASE · AS

EXPECTED OUTPUT COLUMNS

customer_id, customer_name, last_purchase_date, days_since_last_purchase, customer_status

WRITE YOUR SQL HERE

```

SELECT customer_id, customer_name, last_purchase_date,
       DATEDIFF(current_date, last_purchase_date) AS days_since_last_purchase,
       CASE
         WHEN DATEDIFF(current_date, last_purchase_date) < 30 THEN "Active Customer"
         WHEN DATEDIFF(current_date, last_purchase_date) BETWEEN 31 AND 90 THEN "At Risk Customer"
         WHEN DATEDIFF(current_date, last_purchase_date) > 90 THEN "Inactive Customer"
       END AS customer_status
FROM retail_datafunc.customer_recency;

* BETWEEN 31 AND 90 THEN "At Risk Customer"

```